

AXON-RFC-006 附录 A: easynet.pages

有状态 EasyNet 第一个参考状态机的技术设计、算法与实现计划

胡思蓝
新加坡国立大学

草稿 v0.3, April 28, 2026

Abstract

本附录规定 `easynet.pages` ——这是能完整调用 AXON-RFC-006 主文所定义全部规范原语的最简单 `state_type`。一个 `page` 是一个双视图状态对象：它在主文 §1-7 的规则下承载身份、规范内容与版本号，并通过 `human_view`（面向浏览器的 HTML 渲染）与 `agent_view`（包含能力面、EAL 提示与回执历史指针的结构化、机器可执行渲染）同时暴露其状态。本附录覆盖四层细节。第一，状态机：字段集、四种状态以及七个 `transition`。第二，每个 `transition` 参数与回执体的 `wire schema`。第三，算法：canonical projection、JCS 规范化、hash、ETag、乐观并发与视图投影，每条都附字节级 worked example。第四，实现计划：在 `EasyNet-Cli` 内的模块拓扑、五个 PR 切片（P-A1 至 P-A5），以及一个端到端的 worked example——跟踪一个 `page` 从首次 `pages.create`，经 Hub 渲染的 HTML 响应，直到 `agent` 通过 URA 拉取的全过程。附录 B（`easynet.web_app` 压力测试）以及主文 13 条 TR-INV-* 不变量在文中被引用但不被重新定义；本文是一份参考实现规约，不是协议的再推导。

Contents

1	目标与非目标	2
1.1	目标	2
1.2	非目标	2
2	与规范核的关系	3
3	状态机	3
3.1	<code>state_type</code> 与 <code>state_key</code>	3
3.2	规范字段	4
3.3	状态空间	5
3.4	EAL 是 canonical；HTML 与 API 表面是派生	5
3.5	生命周期图	7
4	Wire schemas	7
4.1	<code>pages.create@v1</code>	8
4.2	<code>pages.publish@v1</code>	8
4.3	<code>pages.update@v1</code>	8
4.4	<code>pages.archive@v1</code> , <code>pages.unpublish@v1</code> , <code>pages.delete@v1</code>	9
4.5	<code>pages.get@v1</code> （查询）	9
4.6	<code>pages.list@v1</code> （查询）	9
5	算法	10
5.1	Canonical projection π	10
5.2	Canonical 序列化 σ (JCS)	10
5.3	canonical_state_hash h	10
5.4	字节级 worked example	10
5.5	乐观并发：ETag	11

5.6	视图投影算法	11
5.6.1	human_view (HTML)	11
5.6.2	agent_view (结构化)	12
5.6.3	eal_hint	13
6	架构	13
6.1	模块拓扑 (EasyNet-Cli)	13
6.2	守护进程接线	14
6.3	Hub 接线	14
6.4	存储布局	14
7	实现计划	15
7.1	P-A1: BlobStore + StateStore 骨架	15
7.2	P-A2: pages.* 能力注册 + 回执	15
7.3	P-A3: 视图投影 + EAL 提示	16
7.4	P-A4: Hub 二进制	16
7.5	P-A5: agent_view 增强 + 回执历史 endpoint	16
7.6	Hub envelope 铸造	17
8	端到端 worked example	17
8.1	准备	17
8.2	编辑与发布	17
8.3	浏览器经 Hub 拉取	18
8.4	邻居 agent 经 URA 拉取	18
8.5	审计重放	19
9	暂缓至 RFC-006-A v2 的开放问题	20
9.1	覆盖检查	20

1 目标与非目标

1.1 目标

[规范性] 本附录规定的 easynet.pages 实现需要：

1. 满足 AXON-RFC-006 主文 (§1-§7) 所有 **[规范性]** 条款，并完全支撑配套附录 B 中适用的每一条 TR-INV*。
2. 提供四个规范 transition (create、publish、update、archive、delete) 以及两个查询能力 (get、list)。
3. 将 page 内容存为内容寻址的 blob，以 SHA-256 为键，永远不嵌入规范状态 (CSH-INV-2)。
4. 为每一个 Published page 同时暴露 human_view (从内容 blob 渲染的 HTML) 与 agent_view (包含规范字段、能力面、EAL 提示与回执历史指针的类型化 JSON 文档)。
5. 可以以五个 PR 大小的切片 (§7) 端到端落地。

1.2 非目标

[推论] 以下事项明确不在本附录范围内：

- **webapp**（长任务、运行时叠加压力测试），由附录 B 覆盖。
- **跨 realm 或跨节点的 Hub**。Phase 1 仅交付一个单节点 Hub，反向代理同一守护进程拥有的 pages。多 realm Hub 暂缓至 RFC-006-A v2。
- **已认证人类调用方**（principal/human-auth）。Phase 1 仅支持 principal/human-anon。
- **超出固定默认值的 HTML 净化管线与 CSP 策略**。Phase 1 完全信任 owner 编写的 HTML，Hub 用单一硬编码 CSP。净化/模板层留给 RFC-006-A v2。
- **TLS 终结、限流、多实例 Hub 集群**。Hub 是绑定单端口的 axum 进程；生产级硬化属于运维工作，不属于协议工作。

2 与规范核的关系

本附录是 AXON-RFC-006 主文规范核的一个参考实现。它不重新定义任何原语。映射如下：

主文条款	本附录的实例化
§1 状态对象	§3: pages 的 state_type、key、生命周期。
§1.5 SO-INV-1 不可变	state_key (realm, owner, slug) 在 create 之后不可变。
§2 规范状态哈希	§5: 显式 projection π 、JCS、SHA-256 例子。
§2 CSH-INV-2 内容外置	§6.4: blob 布局；规范状态只持 content_hash。
§3 回执类	§4: 每个 transition 的回执体。
§3 RC-INV-2 乐观并发	§5.5: ETag = 规范哈希前 16 字节。
§4 Transition Ability	§4: 每个 page transition 的 transition_id。
§5 权限 + 委托	Phase 1: owner_agent 本地签名；pages 不需要委托执行。
§5 principal/ caller	§7.6: Hub 按请求铸造 human-anon URI。
§6 视图投影	§5.6: human_view 与 agent_view 算法。
§7 回执索引	§6.4: 除 receipt_id 之外的 state_key 索引。

阅读约定。本附录中带 **[规范性]** 标签的语句仅对 pages 实现有约束力；它不提议新的协议级规则。本文看似在扩展协议时，是在对主文中已经 **[规范性]** 的条款做实例化。

3 状态机

3.1 state_type 与 state_key

[规范性]

state_type: easynet.page

state_key:

(realm: string, owner_agent_uri: string, slug: string)

realm	逻辑命名空间。Phase 1: 每个守护进程一个 realm, 在配置中命名（默认 "default"）。
owner_agent_uri	规范权限。即为本 page 每个 CanonicalReceipt 签名的 agent。按主文 §1.3 嵌在 key 内（不是字段）。
slug	owner 命名空间内的 kebab-case 标识符 ([^] [a-z0-9][a-z0-9-]{0,62}[a-z0-9]\$)。

URA 形态：

```
easynet:///r/<realm>/agent/<owner>/page/<slug>
```

ABNF (slug):

```
slug      = head [ *body tail ]
head      = ALPHA / DIGIT
body      = ALPHA / DIGIT / "-"
tail      = ALPHA / DIGIT
ALPHA     = %x61-7A          ; 仅小写 ASCII 字母
```

slug 语法刻意拒绝大写、点、斜杠以及国际化字符。两条理由：slug 出现在不再做转义的 URL 中；大小写敏感的冲突 (Hello 与 hello) 是行政混乱的常见来源。RFC-006-A v2 可以放宽这条。

3.2 规范字段

[规范性] 按 CSH-INV-2 (主文 §2.5)，大体积内容以哈希持有。所以一个 page 的规范状态是一张小映射：

```
canonical_fields(page) = {
    manifest_hash:    bytes(32),    // manifest 子文档的 SHA-256
                                // (title, description,
                                // content_type, visibility);
                                // 见 §3.3
    eal_hash:         bytes(32),    // page EAL 源代码的 SHA-256
                                // (见 §3.6, PA-INV-2)。agent
                                // 面向的 canonical artifact:
                                // 含 `// ...` 注释中的 markdown
                                // 文档与零或多个 mission 声明。
                                // 在 Draft 与 archive/delete
                                // 之后为 null/zero
    content_hash:     bytes(32),    // 渲染后的 HTML body 的 SHA-256
                                // (即 EAL 源的 human 投影)。
                                // 在 Draft 与 archive/delete
                                // 之后为 null/zero
    state_value:      string,       // "Draft" | "Published"
                                // | "Archived" | "Deleted"
    visibility:       string,       // "private" | "realm" | "public"
    version:          uint64        // 单调；每个 CanonicalReceipt +1
}
```

schema 为每个字段标 canonical:true。以下操作性字段在 Phase 1 存在但绝不进入 canonical projection π (§5)：

```
operational_fields(page) = {
    last_view_at:      timestamp,   // 仅观测
    view_count_estimate: uint64,    // 尽力而为
    last_caller_class: string|null  // "browser" | "bot" | ...
}
```

这些字段从 Hub 在视图投影 (§5.6) 期间发出的 OperationalReceipt 中聚合得到；它们对 agent_view 的审计摘要有用，但永远不进入 canonical_state_hash。

3.3 状态空间

state_value	含义
Draft	已创建但尚未发布。content_hash 可为空或已设；不被 Hub 服务。
Published	已发布。content_hash 非空。若 visibility 允许则被 Hub 服务。
Archived	曾被 Published；不再被 Hub 服务。content_hash 保留以备历史审阅。
Deleted	终态。content_hash 清零。不接受任何后续 transition。

允许的规范 transition（单一规范状态空间，无 runtime 叠加——pages 没有像 web_app 那样的长期运行 serve 进程）：

Transition	Pre → Post	说明
pages.create@v1	$\emptyset \rightarrow \text{Draft}$	铸造 state key。
pages.publish@v1	$\text{Draft} \rightarrow \text{Published}$	要求 content_hash 非空。
pages.update@v1	$\text{Published} \rightarrow \text{Published}$	version+1；content_hash 与/或 manifest_hash 改变。
pages.archive@v1	$\text{Published} \rightarrow \text{Archived}$	可通过重新发布逆转。
pages.unpublish@v1	$\text{Archived} \rightarrow \text{Draft}$	允许恢复编辑。
pages.delete@v1	$* \rightarrow \text{Deleted (终态)}$	清空 content_hash。

查询能力（不改状态、不发 CanonicalReceipt，仅产生用于审计的 OperationalReceipt）：

能力	返回
pages.get@v1	完整 canonical projection + content blob 句柄
pages.list@v1	单 owner 的 (slug, state_value, version) 列表
pages.get_eal@v1	page 的 UTF-8 EAL 源代码（见 §3.6, PA-INV-2）。强制实现。

不变量 — PA-INV-1

处于 Draft 或 Archived 的 page 不允许被任何 Hub 提供，无论 visibility 为何。Hub 在查 visibility 之前先按 state_value 判别；visibility 仅对 Published 起把关作用。

为什么有 PA-INV-1。draft 按定义就是未完成的；仅因 visibility 是 public 就服务它，会把进行中的工作当成已发布内容暴露。Archived page 是有意不被服务的——它们存在是为了历史，而不是访问。

3.4 EAL 是 canonical；HTML 与 API 表面是派生

[规范性] 落在 page 上的邻居 EasyNet agent 想要两件事：理解 page 的内容，以及（当 page 有任何动作时）调用它的 actions。上一稿用两套机制处理（强制的 pages.get_markdown 提供内容；actions.* 命名空间提供调用）。v0.3 把它们统一：page 的 agent 面向 canonical artifact 是单一的 EAL 源文件。

不变量 — PA-INV-2

每个 easynet.page 必须把一份 EAL 源作为 canonical artifact 携带，哈希进 eal_hash。该 EAL 源结构为：// ... 行注释中的 markdown 文档块，后接零或多个 mission 声明，对应 page 的交互元素。对于 state_value = Published 的 page，owner 必须注册并实现 <owner>.pages.get_eal@v1，返回 SHA-256 等于当前 eal_hash 的 UTF-8 EAL 文本。Hub 必须暴露 HTTP 镜像 GET /<...>/easynet-for-agents/eal，把它翻译为 pages.get_eal@v1 调用；响应体是 Content-Type: text/eal (或 text/plain; charset=utf-8 作 fallback)。

规范的撰写方向是 EAL → HTML。owner 写 EAL 源。HTML body 由确定性渲染器从 EAL 生成 (markdown 注释变成渲染过的 HTML；每个 mission 声明绑定到对应 ability 调用的 HTML form / button)。content_hash (HTML body) 与 eal_hash (EAL 源) 都是 canonical 字段，但依赖关系是非对称的：渲染器是 EAL 加 page manifest 的纯函数，所以验证者拿到 EAL 与渲染器版本就能独立重构 content_hash 并与 receipt 对账。

这是 v0.3 的核心断言。page 上的许多 API 与 UI 元素都可以从同一份 EAL 源生成——表单的 action 属性、按钮的 onclick handler、跳转到 action 的 <a> href、page 的 agent 面向摘要全部从同一段 EAL 文本派生。owner 维护一份 canonical artifact；协议保证每个投影都是字节确定性、按内容哈希锚定的。

EAL 源的结构。

```
// # 陶瓷马克杯, 12oz
//
// **价格: ** $19.99
// **发货: ** 周二
//
// 要购买这个杯子, 提交 HTML 表单, 或运行 `buy` mission (EAL) 。

mission "buy" {
  let order = call "shop.checkout" on "alice" with {
    item_id = "mug-12oz",
    address = "<由 owner 端具体化填入>",
    qty      = 1
  } timeout 30
}
```

纯文档 page (无任何交互元素) 携带一个空体 trivial mission:

```
// # 关于本站
//
// 这是一个静态信息 page。这里没有任何动作可做，只有内容供阅读。

mission "view" { }
```

trivial mission "view" { } 满足 EAL parser 当前“每个程序恰好一个 mission”的要求 (src/eal/parser.rs, program = mission_decl)。未来 EAL 放宽以接受零 mission 程序；本附录的契约不依赖此放宽。

HTML/EAL 对应 (提示性, 非严格不变量)。期望 HTML 与 EAL 之间大致信息对等的邻居 agent 能得到很好的服务:

- EAL 注释中的每个 markdown 标题/段落对应到 HTML 中渲染出的同一段内容。
- 每个 mission 声明对应 HTML page 上的一个交互元素 (form / button / 动作链接)；mission 内的 call 与该 form 的 POST /api/<verb> 调用同一个 ability。

- EAL mission 体内引用的每个出站 ability 自动有资格成为 api_surface (web_app, 附录 B); page 声明的 API 由 EAL 派生, 不另行枚举。

协议不强制严格对应 (owner 可以渲染与 EAL 暗示不同的 HTML), 因为 v1 无法机械验证” 这些 HTML 字节是这些 EAL 字节的 canonical 投影”。协议确实强制: 两份字节都作为 canonical 字段存在、都由 owner 签名、两者之间的任何漂移可被对比 EAL 描述行为与 HTML 渲染行为的 peer 检测到。

为什么用单一 EAL 源而非分立 markdown + actions blob。

1. 一份 artifact、一次签名、一个 publish 时刻——markdown 内容与 actions 不能漂移，因为它们物理上是同一段字节。
2. 简单 page 不需要 markdown_hash 字段、不需要单独的 actions.list_for_page 往返：pages.get_eal 一次调用就返回 peer 需要的一切。
3. EAL 已经是 EasyNet 中的 first-class artifact (src/eal/)；为 agent 面向 page 表示复用它，避免发明平行格式。
4. EAL 语法与 peer agent 经 easynet mission run 执行的语法一致，因此”读关于这个 page”与”在这个 page 上做什么”之间的边界塌缩为单一文本 artifact。读取免费；执行 mission 需要 mission run。

与 actions.* (附录 B) 的关系。纯静态 page (除 trivial view 外无 mission) 由其 EAL + pages.get_eal 完全描述; 不需要注册 actions.*。有多个交互元素的 page 或 web_app 同时注册 pages.get_eal@v1 (完整 canonical EAL) 与相关的 actions.* verb (用于带参数具体化的细粒度按 mission 查询)。两者并存: pages.get_eal 是 一次性视图; actions.list_for_page + actions.get(name, params) 是 按需视图。

3.5 生命周期图



4 Wire schemas

每个 transition 的 ability 输入 schema 与其 CanonicalReceipt 的 receipt_schema 列在下面。JSON Schema 简写(仅类型与 required key);规范 schema 落在 abilities/pages.<verb>.ability.toml。

4.1 pages.create@v1

```
args:
{
  slug:          string (匹配 §3.1 slug ABNF, required)
  manifest:      {
    title:        string,          required
    description:   string|null,
    content_type:  "text/html" | "text/markdown",
                                   required
    visibility:    "private" | "realm" | "public",
                                   required
  }
  initial_body:  bytes|null        // 按 content_type 用 utf8 或 base64;
                                   // null 留为空草稿
}
```

CanonicalReceipt 体扩展自主文 §3.1 基底, 新增:

```
{
  page_state_key:    (realm, owner, slug),
  pre_state_hash:    zero(32),      // 创建: pre 为空
  post_state_hash:   sha256(jcs(canonical_fields)),
  state_version:     1,
  manifest_hash:     sha256(jcs(manifest)),
  content_hash:      sha256(initial_body) if initial_body else zero(32),
  blob_handles:      [content_hash] if content_hash != 0 else []
}
```

4.2 pages.publish@v1

```
args:
{
  slug:    string,
  // 可选: 在发布前替换 body
  body:    bytes|null,
  reason:  string|null // 自由文本, 记录在 receipt 中
}
```

precondition: state_value == Draft AND content_hash != 0
(或 `body` 已提供且非空)

CanonicalReceipt 体:

```
{
  page_state_key:    ...,
  pre_state_hash:    canonical_state_hash(Draft 快照),
  post_state_hash:   canonical_state_hash(Published 快照),
  state_version:     prev+1,
  content_hash:      sha256(new body),
  blob_handles:      [content_hash],
  reason:            args.reason
}
```

4.3 pages.update@v1

```
args:
{
  slug:    string,
```



```

manifest:    manifest|null,    // 允许部分更新;
                                //    null = 不变
body:        bytes|null,      // null = 不变
reason:      string|null
}
precondition: state_value == Published AND
              (manifest != null OR body != null)

CanonicalReceipt 体:
{
  page_state_key:    ...,
  pre_state_hash:    h_prev,
  post_state_hash:   h_next,
  state_version:     prev+1,
  manifest_hash:     unchanged or new,
  content_hash:      unchanged or new,
  changed_fields:    ["manifest_hash"|"content_hash", ...],
  reason:            args.reason
}

```

4.4 pages.archive@v1, pages.unpublish@v1, pages.delete@v1

这三个遵循相同模板：小 args (slug, reason)，带正确 pre_state_hash / post_state_hash / state_version 的 CanonicalReceipt。pages.delete@v1 另外清空 content_hash 并递减该 blob 的引用计数（Phase 1 的引用计数是 stub；见 RFC-007）。

4.5 pages.get@v1 (查询)

```

args:
{
  slug:    string,
  version: uint64|null    // null = current
}
returns:
{
  state_key:    ...,
  state_value:  string,
  canonical:    canonical_fields,
  body:         bytes|null,    // 当 caller 无权时
                                // Deleted/Archived 返 null;
                                // 当 visibility 拦下 caller 时
                                // 完全省略

  version:      uint64,
  canonical_state_hash: bytes(32),
  etag:         bytes(16)     // §5.4
}

```

pages.get@v1 产生一个 OperationalReceipt，记录此次读、caller URI 以及（来自 Hub 时）caller_context（主文 §3.6）。

4.6 pages.list@v1 (查询)

```

args:
{
  owner:        string,
  state_filter: ["Draft"|"Published"|...] | null,

```

```

    limit:      uint <= 1000,
    cursor:     string|null
}
returns:
{
  pages:       [{slug, state_value, version, etag, last_modified}],
  next_cursor: string|null
}

```

5 算法

5.1 Canonical projection π

[规范性] 按主文 §2.1, π 是 schema 驱动的, 不能手写。pages 的 schema 列出六个 canonical:true 字段: manifest_hash、eal_hash、content_hash、state_value、visibility、version。 $\pi(v)$ 把 v 投影到这些 key, 按 schema 声明顺序。

```

fn project_pages(state: &PageState) -> Map<String, Value> {
  let mut out = BTreeMap::new();
  out.insert("manifest_hash".into(), bytes_to_hex(&state.manifest_hash));
  out.insert("eal_hash".into(),      bytes_to_hex(&state.eal_hash));
  out.insert("content_hash".into(),  bytes_to_hex(&state.content_hash));
  out.insert("state_value".into(),    state.state_value.clone().into());
  out.insert("visibility".into(),     state.visibility.clone().into());
  out.insert("version".into(),        state.version.into());
  out // BTreeMap 按字典序迭代——确定性
}

```

开销。时间与空间均 $O(1)$ (六个字段、固定大小)。

5.2 Canonical 序列化 σ (JCS)

[规范性] σ 是 RFC 8785 (JSON Canonicalization Scheme)。对 pages 这是直接子集: 对象 key 字典序排序 (已被 BTreeMap 保证)、无多余空白、字节字段用十六进制串、整数无前导零、不出现浮点。pages projection 的 JCS 因此是已投影 JSON 的确定性紧凑形式。

5.3 canonical_state_hash h

[规范性] $h(b) = \text{SHA256}(b)$, Phase 1 固定。

5.4 字节级 worked example

一个处于 Published 状态的 page, 其 canonical projection 如下:

```

{
  "content_hash": "abf0c1...c8f3",    // 32 字节十六进制
  "eal_hash":     "6b21d3...49e0",
  "manifest_hash": "1d4a7e...09e2",
  "state_value":  "Published",
  "version":      3,
  "visibility":   "public"
}

```

JCS 后:

```
{
  "content_hash": "abf0c1...c8f3",
  "eal_hash": "6b21d3...49e0",
  "manifest_hash": "1d4a7e...09e2",
  "state_value": "Published",
  "version": 3,
  "visibility": "public"
}
```

canonical_state_hash = SHA256(上述字节) = 0xa3b91f7e...8d04 (32 字节)。前 16 字节作为 ETag (§5.5)。

5.5 乐观并发：ETag

[规范性] 按 RC-INV-2 (主文 §3.7) 每次规范 transition 必须声明其期望的 pre_state_hash。Hub 与 HTTP 客户端使用 16 字节 ETag 比 32 字节十六进制串方便，所以定义：

$$\text{ETag}(p) = \text{Base64Url}(\text{canonical_state_hash}(p)[0..16])$$

128 位前缀在单 realm Hub 服务的群体规模下抗碰撞安全 (生日界 $\approx 2^{64}$ 个 page 才会发生碰撞)。每次 pages.update, caller 提供 If-Match 头 (或在 EasyNet ability 路径中提供 expected_etag 字段)。回执验证层：

1. 计算 page 当前的 ETag。
2. 与 expected_etag 对比。
3. 不一致：用 ConcurrencyConflict 拒绝该 transition，并在 OperationalReceipt 里记录这次冲突 (这样审计日志保留了被尝试但未提交的内容)。

说明

ETag 是 canonical_state_hash 的展示形式，而非替代品。内部协议逻辑使用完整 32 字节哈希 (RC-INV-2 引用 pre_state_hash)；16 字节 ETag 仅在外部的 HTTP 边界出现。

5.6 视图投影算法

5.6.1 human_view (HTML)

[规范性]

```
fn human_view(p: &PageState, blob: &BlobStore) -> HttpResponse {
  if p.state_value != "Published" {
    return http_404();
  }
  if !visibility_allows(p.visibility, request.caller) {
    return http_404(); // 永远不返 403——会泄漏存在性
  }
  let body = blob.get(&p.content_hash)?;
  let csp = "default-src 'self'; script-src 'none'; \
    style-src 'unsafe-inline' 'self'; img-src 'self' data;; \
    object-src 'none'; base-uri 'none'; frame-ancestors 'none'";
  HttpResponse::ok()
    .content_type(p.manifest.content_type)
    .header("ETag", etag(p))
    .header("Cache-Control", "public, max-age=60, must-revalidate")
    .header("Content-Security-Policy", csp)
    .header("Referrer-Policy", "no-referrer")
    .header("X-Content-Type-Options", "nosniff")
    .body(body)
}
```

确定性。按主文 TR-INV-6, `human_view` 是 (p, blob) 的确定性函数。CSP、缓存头与响应码都是 `p` 与请求 `principal` 的纯函数，与挂钟时间或负载无关。

为什么 `visibility` 不通过时返 404 而非 403。403 会确认 `page` 存在。对 `visibility=private` 这是泄漏。该折中（私有 `page` 与不存在的 `page` 不可区分）是有意为之。

5.6.2 `agent_view` (结构化)

[规范性]

```
fn agent_view(p: &PageState) -> Value {
  let surface = derive_ability_surface(p); // VP-INV-1
  let history = receipt_history_pointer(p); // §6.3
  json!({
    "state_type": "easynet.page",
    "state_key": { "realm": p.realm,
                  "owner": p.owner,
                  "slug": p.slug },
    "ura": p.ura(),
    "canonical": {
      "state": p.state_value,
      "manifest": p.manifest, // 小, 可内联
      "content_hash": hex(p.content_hash),
      "visibility": p.visibility,
      "version": p.version,
      "etag": etag(p)
    },
    "ability_surface": surface,
    "history": {
      "versions": p.version,
      "latest_canonical_receipt_id": p.last_receipt_id,
      "receipt_log_ura": history
    },
    "eal_hint": eal_hint_for(p), // §5.6.3
    "view_meta": {
      "generated_from": "(canonical, runtime=, caller_context)"
    }
  })
}

fn derive_ability_surface(p: &PageState) -> Vec<&'static str> {
  match p.state_value.as_str(){
    "Draft" => vec!["pages.publish@v1", "pages.update@v1",
                  "pages.delete@v1"],
    "Published" => vec!["pages.update@v1", "pages.archive@v1",
                      "pages.delete@v1"],
    "Archived" => vec!["pages.unpublish@v1", "pages.delete@v1"],
    "Deleted" => vec![],
    - => vec![],
  }
}
```

符合 VP-INV-1。`ability_surface` 严格从 `p.state_value` 与静态 transition schema (match 块)派生。它不是能力注册表的快照；某个能力虽然已注册但其 `pre_state` 不包括 `p.state_value`，则在此处不可见。

5.6.3 eal_hint

[规范性] eal_hint 是单行 EAL 片段，邻居 agent 可复制粘贴来推进 page 从当前状态出发。例：

```
Draft      -> "call pages.publish on <owner> with { slug: \"<s>\" } timeout 30"
Published -> "call pages.update on <owner> with { slug: \"<s>\", body: <new> } timeout
60"
Archived  -> "call pages.unpublish on <owner> with { slug: \"<s>\" } timeout 30"
Deleted   -> ""      // 无 EAL 提示；surface 为空
```

提示生成器是确定性的；agent 在没有任何 transition 介入的情况下两次读取 URA，得到的提示字节完全一致。

6 架构

6.1 模块拓扑 (EasyNet-Cli)

src/runtime/state/ mod.rs store.rs hash.rs receipt.rs	← 新增 (P-A1, P-A2) pub use 下面的类型 StateStore trait + sled 实现 project / canonicalize_jcs / sha256 CanonicalReceipt, OperationalReceipt
src/runtime/blob_store/ mod.rs fs.rs	← 新增 (P-A1) BlobStore trait FsBlobStore (内容寻址目录树, sha256 为 key)
src/runtime/agents/pages_ability.rs	← 新增 (P-A2) register: create/publish/update/ archive/unpublish/delete/get/list
src/runtime/views/ mod.rs human.rs agent.rs	← 新增 (P-A3) View trait human_view (CSP, blob 拉取) agent_view (JSON 投影)
src/bin/easynet-hub.rs	← 新增 (P-A4) axum 二进制；path → URA → view
src/runtime/hub/ mod.rs router.rs caller.rs	← 新增 (P-A4) path 解析、view 选择 principal/human-anon URI 铸造 + caller_context 抽取
src/runtime/eal_hint.rs	← 新增 (P-A5) 确定性 EAL 模板

设计选择：state 与 receipt 索引用 sled。sled 已经是相邻代码的依赖；它支持范围查询（满足主文 §7.1 要求的 state_key 前缀扫描）以及原子批写（用于把 canonical receipt 与 state mutation 同时落盘）。RFC-006 主文不绑定后端；如果将来 sled 被替换（例如换成 SQLite），StateStore trait 不变。

设计选择：文件系统 blob store。Phase 1 把 blob 存在 <state-root>/blobs/<aa>/<bb>/<full-hash>，其中 aa 与 bb 是 SHA-256 的前两个十六进制字节。两层 fan-out 让任一目录在大规模下也保

持在几千个条目以内。RFC-007 会用引用计数与 GC 扫描细化它；Phase 1 两者都 stub (Phase 1 永不删除 blob)。

6.2 守护进程接线

启动时守护进程：

1. 在 `$EASYNET_HOME/state.sled` 打开 state store。
2. 在 `$EASYNET_HOME/blobs/` 打开 blob store。
3. 对注册表中每个 agent 调用 `pages_ability::register`；注册把 (state_store, blob_store, agent_name) 捕获到 handler 闭包内。
4. 启动 receipt 索引 (基于同一 sled tree 的 state_key 前缀索引)。
5. 恢复任何在途 transition (pages 没有：每个 transition 都是单步的，没有 Building 这种中间态)。

6.3 Hub 接线

独立二进制 easynet-hub，内嵌 axum 路由：

```
GET /<realm>/<owner>/<slug>          -> human_view
GET /<realm>/<owner>/<slug>/agent     -> agent_view (JSON)
GET /<realm>/<owner>/<slug>/receipts -> 回执历史 (JSON)
GET /healthz                          -> 200 "ok"
```

Hub 是守护进程的客户端：它不直接访问 state store。每次视图拉取都是一个对守护进程的 `pages.get@v1` 调用，envelope 按主文 §5.4 填 (caller URI 为 `principal/human-anon/...`，caller_context 按主文 §3.6 填)。

信任边界。 Hub 被允许铸造 `principal/human-anon` caller，但不能铸造任意 `agent/` caller。守护进程在准入时校验 caller URI；如果一个 `agent/` URI 来自 Hub 但不匹配 Hub 的授权令牌 (Phase 1 是带外共享 secret；RFC-008 会替换为正规签名)，则拒绝。

6.4 存储布局

```
$EASYNET_HOME/
state.sled/          sled 数据库
trees:
  "page::canonical"   key: jcs((realm,owner,slug))
                     value: bincode(PageState)
  "page::receipts"    key: (jcs((realm,owner,slug)),
                     version_be_u64,
                     receipt_id)
                     value: bincode(CanonicalReceipt)
  "page::idx_owner"   key: (owner, slug)
                     value: state_key (反向指针)
  "page::idx_receipt_id" key: receipt_id
                     value: state_key
blobs/
  ab/
    f0/
      abf0c1...c8f3  原始字节
  ...
```

两个索引(idx_owner、idx_receipt_id)满足主文 §7.1 的双轴枚举要求:回执日志按 receipt_id (保留原 EasyNet invocation 可追踪性) 与按 state_key (新的对象粒度可追踪性) 都可枚举。

7 实现计划

五个 PR 切片。每个切片可独立审阅与验证; 先合并第一个不会阻塞其余切片在并行分支上推进。

7.1 P-A1: BlobStore + StateStore 骨架

新增文件。src/runtime/blob_store/ (mod.rs + fs.rs); src/runtime/state/ (mod.rs + store.rs + hash.rs + receipt.rs)。

公开 API。

```
pub trait BlobStore: Send + Sync {
    fn put(&self, bytes: &[u8]) -> Result<[u8; 32]>;           // 返回 sha256
    fn get(&self, hash: &[u8; 32]) -> Result<Vec<u8>>;
    fn exists(&self, hash: &[u8; 32]) -> Result<bool>;
}

pub trait StateStore<S: StateObject>: Send + Sync {
    fn read(&self, key: &S::Key) -> Result<Option<S>>;
    fn apply(
        &self,
        receipt: &CanonicalReceipt,
        post_state: &S,
    ) -> Result<()>;      // 原子: state mutation + receipt append
    fn list_by_owner(...) -> Result<Vec<S::Key>>;
    fn enumerate_receipts(&self, key: &S::Key)
        -> Result<Vec<CanonicalReceipt>>;
}

pub fn project(state: &dyn StateObject) -> BTreeMap<String, Value>;
pub fn canonicalize_jcs(map: &BTreeMap<String, Value>) -> Vec<u8>;
pub fn canonical_state_hash(state: &dyn StateObject) -> [u8; 32];
```

测试。JCS 一致性 (一组 RFC 8785 测试向量); SHA-256 往返; 并发尝试下 StateStore::apply 的原子性 (钉住 RC-INV-2 不变量)。

验收。cargo test -p easynet --lib state 通过; 基准测试在普通硬件上 $\geq 50k$ 次/秒规范哈希 (合理性下界, 不是严格要求)。

7.2 P-A2: pages.* 能力注册 + 回执

新增文件。src/runtime/agents/pages_ability.rs。触及 src/runtime/agents/mod.rs 来注册 namespace。

行为。§4 列出的每个 transition 都实现:

1. 用 ability schema 校验 args。
2. 通过 StateStore 读当前 state。
3. 校验 pre_state_hash (RC-INV-2)。
4. 计算 post_state 与 post_state_hash。

5. 构建 CanonicalReceipt; 用 owner 私钥签名 (Phase 1: 用每个 agent 的硬编码 keypair stub Ed25519 签名; 密钥管理推迟至 RFC-008)。
6. StateStore::apply (原子)。
7. 返回结果。

测试。

- 每个 transition 往返。
- 拒绝从 Archived 出发的 publish (必须先 unpublish)。
- 拒绝带不合规 slug 的 create。
- 拒绝在 Deleted 上的任何 transition (终态)。
- 乐观并发: 两个针对同一 page slug 并发的 update; 一个赢, 一个被 ConcurrencyConflict 拒绝。

验收。该 ability 可经 easynet ability invoke 调用; 回执经现有回执日志面可见。

7.3 P-A3: 视图投影 + EAL 提示

新增文件。src/runtime/views/ + src/runtime/eal_hint.rs。

行为。纯函数: 一个 PageState 映射到 (human_view_response, agent_view_json) 二元组加 EAL 提示字符串。无网络、无 state 改动。

测试。确定性 (同一输入 1000 次产出相同字节); ability_surface 在每个状态下匹配 §3.3; CSP 头恒定。

7.4 P-A4: Hub 二进制

新增文件。src/bin/easynet-hub.rs + src/runtime/hub/。

行为。axum 路由, 三个 GET 路由, 每条:

1. 解析 path $\rightarrow$ (realm, owner, slug) $\rightarrow$ state_key。
2. 构 envelope: caller URI 为 principal/human-anon/<ip-hash>/<session-id> (§7.6); caller_context 从 HTTP 头填。
3. 调用守护进程的 pages.get@v1。
4. 应用视图投影 (P-A3)。
5. 返回 HTTP 响应。

测试。集成: 在临时目录里启动一个守护进程 + 一个 Hub, create+publish 一个 page, curl 经 Hub 访问, 断言 HTML 响应 + 正确 ETag + 正确 CSP。curl 后缀 /.agent, 断言 JSON 形态匹配 §5.6。

7.5 P-A5: agent_view 增强 + 回执历史 endpoint

行为。实现 /.receipts 路由: 返回某 page 的回执历史 (仅 canonical receipts, 按 version 顺序) 作为 JSON 数组。给 agent_view 加上 eal_hint 字段 (在 P-A3 中已规定但仅 stub)。

测试。客户端用 canonical_state_hash 重构来重放回执历史; 断言最终 hash 等于 page 当前 hash。

7.6 Hub envelope 铸造

```
fn build_envelope(req: &HttpRequest) -> Envelope {
  let ip    = client_ip(req);
  let salt = HUB_LOCAL_SALT.read();
  let ip_hash = hmac_sha256(&salt, ip.to_string().as_bytes())
    .iter().take(8)
    .flat_map(|b| format!("{:02x}", b).chars().collect::<Vec<_>>())
    .collect::<String>();
  let sid = req.cookie("easynet_session")
    .or_else(|| req.header("X-EasyNet-Session"))
    .or_else(|| req.query("session"))
    .unwrap_or_else(|| "-".to_string());

  let caller = format!(
    "easynet:///principal/human-anon/{ip_hash}/{sid}"
  );
  let caller_context = json!({
    "origin": {
      "referrer_url": req.header("Referer"),
      "referrer_realm": referer_origin(req.header("Referer")),
      "direct": is_direct_navigation(req)
    },
    "agent_class": classify_user_agent(req.header("User-Agent")),
    "headers": {
      "accept_language": req.header("Accept-Language"),
      "sec_fetch_site": req.header("Sec-Fetch-Site"),
      "sec_fetch_mode": req.header("Sec-Fetch-Mode"),
      "sec_fetch_dest": req.header("Sec-Fetch-Dest"),
      "x_forwarded_for_hash": xff_hash(req)
    }
  });
  Envelope { caller, caller_context, ... }
}
```

8 端到端 worked example

从 `pages.create` 开始，经 Hub 渲染的 HTML 访问与 agent URA 拉取，完整跟踪。

8.1 准备

```
$ easynet agent add alice --type claude-code
$ easynet-daemon &          # 在 $EASYNET_HOME ipc 上监听
$ easynet-hub --bind 127.0.0.1:8787 &
```

8.2 编辑与发布

```
$ easynet ability invoke alice.pages.create --args '{
  "slug": "shop-001",
  "manifest": {
    "title": "陶瓷马克杯, 12oz",
    "content_type": "text/html",
    "visibility": "public"
  },
  "initial_body": "<h1>Mug</h1><p>$19.99 — 周二发货.</p>"
}'
```

```

response:
  receipt_id:      "r_a1b2..."
  state_value:     "Draft"
  canonical_state_hash: "0xa3b91f7e..."
  version:         1

$ easynet ability invoke alice.pages.publish --args '{ "slug": "shop-001" }'

response:
  receipt_id:      "r_c3d4..."
  state_value:     "Published"
  canonical_state_hash: "0x4d28ee17..."
  version:         2
  etag:            "TSjuFwOpAvjpAQ=="

```

8.3 浏览器经 Hub 拉取

```

$ curl -i http://127.0.0.1:8787/default/alice/shop-001

HTTP/1.1 200 OK
ETag: "TSjuFwOpAvjpAQ=="
Cache-Control: public, max-age=60, must-revalidate
Content-Security-Policy: default-src 'self'; script-src 'none'; ...
Content-Type: text/html
Content-Length: 49

<h1>Mug</h1><p>$19.99 — 周二发货。</p>

```

Hub 为该读发出一条 OperationalReceipt:

```

{
  "receipt_class": "operational",
  "state_key":     ["default", "alice", "shop-001"],
  "transition_id": "pages.get@v1",
  "attempt_id":    "att_e5f6...",
  "envelope": {
    "caller": "easynet:///principal/human-anon/9c4e.../sess-abc",
    "caller_context": {
      "origin": { "referer_url": null,
                  "referer_realm": null,
                  "direct": true },
      "agent_class": { "kind": "browser",
                       "user_agent_hash": "a1b2..." },
      "headers": { "accept_language": "zh-CN",
                   "sec_fetch_site": "none",
                   "sec_fetch_mode": "navigate",
                   "sec_fetch_dest": "document" }
    }
  },
  "observed_at": "2026-04-28T10:14:32Z"
}

```

8.4 邻居 agent 经 URA 拉取

```

$ curl http://127.0.0.1:8787/default/alice/shop-001/.agent | jq

```

```
{
  "state_type": "easynet.page",
  "state_key": { "realm": "default",
                 "owner": "alice",
                 "slug": "shop-001" },
  "ura": "easynet:///r/default/agent/alice/page/shop-001",
  "canonical": {
    "state": "Published",
    "manifest": { "title": "陶瓷马克杯, 12oz",
                  "content_type": "text/html",
                  "visibility": "public" },
    "content_hash": "0x4d28ee17...",
    "visibility": "public",
    "version": 2,
    "etag": "TSjuFw0pAvjpAQ=="
  },
  "ability_surface": ["pages.update@v1", "pages.archive@v1",
                     "pages.delete@v1"],
  "history": {
    "versions": 2,
    "latest_canonical_receipt_id": "r_c3d4...",
    "receipt_log_ura": "easynet:///r/default/agent/alice/page/shop-001/receipts"
  },
  "eal_hint": "call pages.update on alice with { slug: \"shop-001\", body: <new> } timeout 60"
}
```

8.5 审计重放

邻居拉取回执历史并在本地重放：

```
$ curl http://127.0.0.1:8787/default/alice/shop-001/.receipts | jq -r '.[].
  post_state_hash'
0xa3b91f7e... # version 1, create 之后
0x4d28ee17... # version 2, publish 之后
```

邻居：

1. 按顺序从每个回执重构 canonical state。
2. 为每次重构计算 canonical_state_hash。
3. 断言每次计算的 hash 等于回执的 post_state_hash。
4. 断言最终 hash 等于 agent_view 的 canonical.content_hash（严格说不直接如此——比对在 canonical_state_hash 上做，不过 content_hash 是它的输入之一）。
5. 用 alice 公开的密钥校验每条回执的 owner_signature。

审计成功的含义：邻居有密码学证据证明这个 page 在 version 2 所呈现的内容正是这些字节，由 Alice 签名，时间不晚于最终回执的时间戳。这是传统 web 讲不出的审计故事。

9 暂缓至 RFC-006-A v2 的开放问题

1. **HTML 净化**。Phase 1 完全信任 owner 写的 HTML (CSP `script-src 'none'` 覆盖最坏情况, 但 `onerror=` 式属性注入仍可穿透)。v2 应规定 publish 时运行的安全子集净化器并对其输出 hash 进行钉死。
2. **多 realm Hub**。服务多于一个 realm 的 Hub 需要路由表; 路径方案 `/<realm>/...` 已经容纳, 但 Phase 1 在启动时绑定单 realm。
3. **Page 模板 / 槽位填充**。常见 page (产品目录、博客文章) 结构重复。模板层可让 owner 填 manifest + 槽位数据, 使 `content_hash` 覆盖渲染输出。
4. **已认证人类 (principal/human-auth)**。对需要真实用户身份的 private/realm page, 必须规定 `principal/human-auth/<provider>/<user-id>` 子命名空间及其签名机制。
5. **联邦读取**。由节点 A 拥有的 page 经节点 B 上的 Hub 被读取。Hub 由守护进程客户端升级为联邦客户端。RFC-006-A v2。

9.1 覆盖检查

主文规则	由谁兑现	位置
SO-INV-1 (key 不可变)	state_key 解析器拒绝 key 更新	§3
CSH-INV-1 (hash 取自 projection)	project_pages + JCS only	§5
CSH-INV-2 (内容外置为 blob)	FsBlobStore; 规范状态只持哈希	§6.4
RC-INV-1 (receipt 绑定)	每条回执均带 (state_key, transition_id, attempt_id)	§4
RC-INV-2 (乐观并发)	在回执校验时间检查 ETag	§5.5
TA-INV-1 (必须带 transition_id)	每个 pages.<verb>@v1 字符串均被校验	§4
VP-INV-1 (ability_surface 来自 state)	derive_ability_surface match 块	§5.6
TR-INV-7 (multi-view 一等公民)	human_view + agent_view 均已实现	§5.6
TR-INV-11 (URA-receipt 可追踪)	/.receipts endpoint + idx_owner 索引	§6.4, §8
TR-INV-12 (principal/human-anon)	build_envelope 铸造该 URI	§7.6
TR-INV-13 (caller_context)	由 build_envelope 填充; 进入 operational receipt	§7.6, §8

结语。 pages 是触发有状态 EasyNet 完整规范核的最简单参考状态机。落地上述五个 PR 完成有状态 EasyNet 的 Phase 1, 并解锁压力测试状态机 (webapp, 附录 B) 以及之后的 agent-native web 层。

本文档是一份 Phase 1 实现规约。与 AXON-RFC-006 主文不一致的地方是本文档的 bug; 与附录 B 压力测试不一致的地方可能是本文档或主文的 bug, 需在对齐轮中调和。